

Library of Image Processing Functions and Algorithms

Computer Vision Final Project

May 2026

Authors:

Adham Yousry Abdulwanis — ID: 23012043
Sohayl Islam Anwar — ID: 23012045
Ahmed Mahmoud El-Saied — ID: 23011215
Kirollos Monir Roshdy — ID: 23012081

Faculty of Computers and Data Science
Alexandria University
2025

02-24-03305 : Computer Vision

Contents

1	Selective Object Enhancement	3
1.1	Goal	3
1.2	Approach Overview	3
1.3	Design	3
1.4	Implementation Details	3
1.4.1	<i>Color Detection Using HSV</i>	3
1.4.2	<i>Mask Generation</i>	3
1.4.3	<i>Enhancement Effects</i>	3
1.5	Results and Observations	3
2	Sharpening X-Ray	4
2.1	Goal	4
2.2	Approach Overview	4
2.3	Design	4
2.4	Implementation Details	4
2.4.1	<i>Edge Detection</i>	4
2.4.2	<i>Smoothing</i>	4
2.4.3	<i>Mask Creation</i>	4
2.4.4	<i>Sharpening</i>	5
2.4.5	<i>Intensity Transformations</i>	5
2.5	Results and Observations	5
2.5.1	<i>Sobel + Gamma</i>	5
2.5.2	<i>Laplacian + Log</i>	5
2.5.3	<i>Canny + Sigmoid</i>	5
2.5.4	<i>Recommendation</i>	5
3	Auto Image Enhancement System	6
3.1	Goal	6
3.2	Implementation	6
3.3	Enhancement Cases	6
3.3.1	<i>Overexposed Image (Mean $L > 160$)</i>	6
3.3.2	<i>Underexposed Image (Mean $L < 90$)</i>	6
3.3.3	<i>Normally Exposed Image ($90 \leq \text{Mean } L \leq 160$)</i>	6
3.4	Final Step	6
4	Document Cleaning System	7
4.1	Median Filtering	7
4.2	EDE Method (Edge Detection, Dilation & Erosion)	7
4.3	Adaptive Thresholding	7
5	Panorama Image Stitching	9
5.1	What is Panorama Image Stitching?	9
5.2	Pipeline	9
5.2.1	<i>Step 1 — SIFT Feature Extraction (<code>find_feature_matches</code>)</i>	9
5.2.2	<i>Step 2 — FLANN Matching and Ratio Test (<code>find_feature_matches</code>)</i>	9
5.2.3	<i>Step 3 — Homography Estimation and RANSAC (<code>compute_homography</code>)</i>	9
5.2.4	<i>Step 4 — Perspective Warping and Alpha Blending (<code>warp_and_blend</code>)</i>	9
5.2.5	<i>Step 5 — Bounding Box Cropping (<code>crop_black_borders</code>)</i>	9
6	Transformed Object Recognition	10

6.1	Goal	10
6.2	Pipeline	10
7	Depth Approximation from Two Images	11
7.1	Pipeline	11
7.1.1	1. Image Loading and Resizing	11
7.1.2	2. Image Alignment (Registration)	11
7.1.3	3. Preprocessing	11
7.1.4	4. Disparity Estimation	11
7.1.5	5. Normalisation	11
8	High Dynamic Range (HDR)	12
8.1	What is HDR?	12
8.2	Pipeline	12
8.2.1	Step 1 — Exposure Bracketing	12
8.2.2	Step 2 — HDR Merge	12
8.2.3	Step 3 — Tone Mapping (Reinhard)	12

1 Selective Object Enhancement

1.1 Goal

Selectively enhancing objects in an image according to their color.

1.2 Approach Overview

The system works in four steps:

1. Detects a target color using HSV color segmentation.
2. Applies one enhancement effect to the detected object.
3. Applies a different effect to the background.
4. Combines both results into a final processed image.

1.3 Design

Three modular functions were used for this task:

- `build_mask()` — Detects the target color and creates a mask.
- `apply_enhancement()` — Applies image enhancement effects.
- `selective_enhance()` — Controls the full enhancement pipeline.

1.4 Implementation Details

1.4.1 Color Detection Using HSV

HSV is used to separate the hue, saturation, and value components. A set of defined HSV color ranges is specified for whichever image is used.

1.4.2 Mask Generation

1. Converts the image from BGR to HSV.
2. Thresholds the target color using `cv2.inRange()`.
3. Removes noise using morphological operations.
4. Smooths the mask using Gaussian Blur.

1.4.3 Enhancement Effects

Gaussian blur, sharpening, and grayscale effects are applied selectively based on the generated mask.

1.5 Results and Observations

Because of the chosen parameters in HSV, the red flower was sharpened while the yellow flower was blurred in the background.

2 Sharpening X-Ray

2.1 Goal

Compare three different sharpening pipelines to determine which is best suited for X-ray medical images.

2.2 Approach Overview

The system:

- Detects edges using different edge detectors.
- Constructs enhancement masks from edge information.
- Sharpens image details.
- Applies intensity transformation techniques to improve visual quality.

Enhancement pipelines used: **Sobel**, **Laplacian**, and **Canny**, each combined with **Gamma**, **Logarithmic**, and **Sigmoid** intensity transforms respectively.

2.3 Design

- `sobel()` — Edge detection using image gradients.
- `laplacian()` — Second derivative edge detection.
- `canny()` — Multi-stage edge detection.
- `smooth()` — Noise reduction and edge smoothing.
- `sharpen()` — Adds enhancement mask to image.
- `gamma()` — Gamma intensity correction.
- `log_transform()` — Logarithmic brightness enhancement.
- `sigmoid()` — Contrast enhancement using sigmoid mapping.
- `SharpeningPipeline` — Main configurable enhancement system.

2.4 Implementation Details

2.4.1 Edge Detection

Identifies regions containing object boundaries, texture details, and high-frequency information.

2.4.2 Smoothing

After edge detection, smoothing is applied to reduce noise and soften sharp transitions. Two types of filters are used: Average and Gaussian.

2.4.3 Mask Creation

The mask is generated by:

$$M = (\text{Smooth})^2$$

This formula emphasizes stronger edges while suppressing weaker details and noise.

2.4.4 Sharpening

Done by adding the mask back to the original image, enhancing edges, fine textures, and overall image clarity.

2.4.5 Intensity Transformations

After sharpening, one of three transformations is applied:

- **Gamma** correction.
- **Logarithmic** brightness enhancement.
- **Sigmoid** contrast mapping.

2.5 Results and Observations

2.5.1 Sobel + Gamma

1. Smooth and balanced sharpening.
2. Preserves anatomical structure naturally.
3. Brightness is increased gently.
4. Less aggressive than Laplacian.

2.5.2 Laplacian + Log

1. Produces strong detail enhancement.
2. Preserves more grayscale information than Canny.
3. Bone structures become clearer while keeping smooth transitions.
4. Bright regions become more emphasized.

2.5.3 Canny + Sigmoid

1. Produces very sharp and thin edges.
2. Skeleton boundaries become highly visible.
3. Background becomes almost completely black.
4. Image appears highly contrasted.

2.5.4 Recommendation

Sobel + Gamma is the best pipeline because it preserves the most natural appearance, retains grayscale anatomy, shows less artifact amplification, and is more robust to noise compared to Laplacian.

3 Auto Image Enhancement System

3.1 Goal

Automatic image enhancement for exposure correction and low-contrast images.

3.2 Implementation

A single main method `enhance()` takes an image as input. The image is converted into the LAB colour space and split into three channels; only the **L channel** (luminance) is processed, while the A and B channels are preserved and merged back at the end.

The method measures the average luminance value across all pixels as a brightness indicator.

3.3 Enhancement Cases

3.3.1 Overexposed Image (Mean L > 160)

1. The image is scaled linearly across all pixels to approximately 120 luminance.
2. A gamma of 1.5 is then applied to non-linearly compress bright spots and recover highlight detail.

3.3.2 Underexposed Image (Mean L < 90)

A gamma of 0.5 is applied, which enhances darker spots more than bright ones, revealing shadow detail.

3.3.3 Normally Exposed Image ($90 \leq \text{Mean L} \leq 160$)

CLAHE (Contrast Limited Adaptive Histogram Equalization) is applied. It divides the image into separate tiles and applies histogram equalization to each tile independently, enhancing contrast without significantly altering exposure.

3.4 Final Step

After whichever case is applied, the new luminance map is merged back with the A and B LAB channels, and the image is converted back into BGR.

4 Document Cleaning System

Three different techniques are implemented to clean dirty areas in documents:

1. Median Filtering
2. EDE Method (Edge Detection, Dilation & Erosion)
3. Adaptive Thresholding

4.1 Median Filtering

Median filtering is the simplest denoising technique and follows two basic steps:

1. Obtain the “background” of an image using Median Filtering with a kernel size of 23×23 .
2. Subtract the background from the image, leaving only the “foreground” (text and significant document details) clean of any background noise.

Implementation notes:

- `cv2.absdiff()` is used instead of `cv2.subtract()` to calculate the absolute difference, ensuring all edges, text, and objects remain visible.
- `cv2.normalize()` maps the darkest pixel to 0 and the brightest to 255.
- `cv2.bitwise_not()` is applied to invert the result to the correct polarity.

4.2 EDE Method (Edge Detection, Dilation & Erosion)

Before applying this technique, images are preprocessed to remove edge noise.

1. **Dilation** makes lines thicker by adding pixels to boundaries, “filling in” the text while keeping stain edges hollow.
2. **Erosion** (the reverse operation) completely removes thin lines while preserving the thicker text lines.

Implementation notes:

- Standard Canny produced an entirely black image, so **Auto-Canny** is used to automatically calculate the appropriate lower and upper thresholds.
- A closing pipeline (Dilation followed immediately by Erosion) is then applied. Multiple kernel sizes were tested to reach an optimal result.

4.3 Adaptive Thresholding

Thresholding sets all pixels above a threshold to 1 (background) and the remaining pixels to 0 (foreground). Adaptive thresholding computes a per-pixel threshold rather than a single global value, using **Gaussian Thresholding**: the threshold is the weighted sum of neighbouring pixel intensities with Gaussian-weighted windows.

Key parameters:

- **Block Size (7)**: Determines the size of the pixel neighbourhood. Larger values provide more global context; smaller values capture fine details.

- **C (23)**: A fine-tuning buffer to help filter out residual noise.

5 Panorama Image Stitching

5.1 What is Panorama Image Stitching?

Panorama stitching is the process of combining two or more photos with overlapping views to produce a high-resolution, seamless panoramic image. The goal is to align photos geometrically and blend them photometrically so that the seam between them is invisible, creating a single continuous image.

5.2 Pipeline

5.2.1 Step 1 — SIFT Feature Extraction (`find_feature_matches`)

The SIFT algorithm identifies keypoints that are invariant to scale and rotation. Each keypoint is converted into a 128-dimensional descriptor (a numerical fingerprint). It uses a Difference of Gaussians (DoG) to find stable blobs across different image scales:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

5.2.2 Step 2 — FLANN Matching and Ratio Test (`find_feature_matches`)

Instead of brute-force comparison, FLANN uses randomised KD-Trees to find the two nearest neighbours (d_1, d_2) for each descriptor. Only matches satisfying the ratio test are kept:

$$\frac{\text{Distance}(d_1)}{\text{Distance}(d_2)} < 0.7$$

5.2.3 Step 3 — Homography Estimation and RANSAC (`compute_homography`)

A **Homography Matrix** H (3×3) maps the projection of points from one image plane to another. RANSAC iteratively selects 4 random point correspondences to build a model and discards outliers that do not fit the geometric consensus.

5.2.4 Step 4 — Perspective Warping and Alpha Blending (`warp_and_blend`)

The second image is warped into the perspective of the first using `cv2.warpPerspective`. Alpha blending is applied to the overlapping region to hide the seam.

5.2.5 Step 5 — Bounding Box Cropping (`crop_black_borders`)

Warping creates a large canvas with empty black areas. A binary threshold detects all non-zero pixels, and `cv2.boundingRect` calculates the coordinates (x, y, w, h) of the tightest rectangle enclosing all content, which is then cropped out.

6 Transformed Object Recognition

6.1 Goal

Recognise an object in a scene image even when it has undergone geometric transformations such as rotation, scaling, or perspective change.

6.2 Pipeline

1. Both images are converted to grayscale and fed to the **SIFT** algorithm, which returns a list of keypoints (distinct features) alongside a descriptor for each keypoint (a unique fingerprint).
2. If no descriptors are found in either image, SIFT returns **None** and the process stops.
3. Otherwise, a matcher takes each descriptor from the reference image and finds the two closest matches in the scene image. A match is only kept if the best match is clearly better than the second-best; ambiguous matches are discarded.
4. If at least **4 good matches** are found, their coordinates are extracted from both images and passed to the homography estimator.
5. The **RANSAC** algorithm identifies geometrically consistent matches and outputs a mask labelling each match as inlier or outlier.
6. Outlier matches are removed during the drawing stage.
7. The final output shows both images side by side with **green** lines connecting the surviving verified matches.
8. If fewer than 4 matches are found, the mask is considered null and surviving matches are drawn without any geometric filtering.

7 Depth Approximation from Two Images

7.1 Pipeline

7.1.1 1. Image Loading and Resizing

After loading the left and right images, their dimensions are compared. If they differ, the right image is resized to match the left image to ensure correct pixel-wise comparison.

7.1.2 2. Image Alignment (Registration)

Keypoints are detected in both images and matched using a **Brute-Force Matcher**. An affine transformation is then computed to horizontally align objects in the right image with the left image, which is necessary for accurate disparity calculation.

7.1.3 3. Preprocessing

Both images are converted to grayscale. The absolute difference between them is computed to visualise the quality of alignment.

7.1.4 4. Disparity Estimation

The **StereoSGBM** (Semi-Global Block Matching) algorithm from OpenCV is applied. Disparity refers to the horizontal shift of an object between the two images:

- Objects **closer to the camera** produce a **larger shift** (higher disparity).
- Objects **farther away** produce a **smaller shift** (lower disparity).

7.1.5 5. Normalisation

Negative disparity values are normalised to a standard 0–255 scale:

- **White (255)** represents objects closest to the camera.
- **Black (0)** represents objects farthest away.

8 High Dynamic Range (HDR)

8.1 What is HDR?

HDR (High Dynamic Range) is a technique that combines multiple photos of the same scene taken at different exposure levels into a single image. The goal is to capture detail in both very dark (shadow) and very bright (highlight) areas that a standard camera cannot capture in a single shot due to its limited dynamic range.

8.2 Pipeline

8.2.1 Step 1 — Exposure Bracketing

Two versions of the same image are created at different exposure levels: **dark** ($\times 0.2$) and **bright** ($\times 4.0$). Each version captures different information:

- The dark image preserves highlight detail.
- The bright image reveals shadow detail that was previously invisible.

8.2.2 Step 2 — HDR Merge

The exposures are merged into a single floating-point radiance map. Each pixel is assigned a weight based on how well-exposed it is. Mid-tone pixels (around value 127) receive the highest weight, while clipped highlights (near 255) and crushed shadows (near 0) receive low weight. The weighting function is:

$$w = 1 - \left(\frac{\text{pixel}}{127.5} - 1 \right)^2$$

8.2.3 Step 3 — Tone Mapping (Reinhard)

The merged HDR radiance map contains floating-point values far beyond the displayable 0–255 range. The **Reinhard operator** compresses this wide range back into a displayable image:

$$L_{\text{out}} = \frac{L}{1 + L}$$

This curve gently compresses bright values while leaving darker values relatively untouched, preserving natural contrast and avoiding the over-brightening seen in simpler methods.

Thank You!